SnapServe / VoiceOrch — Architect Handover Document

> *Last updated: June 2026*
> *Production URL: https://app.snapserve.ai*
> *GitHub repo: https://github.com/srundme/voiceorch (branch: main)*
> *Deployed on: Railway (auto-deploys from GitHub main)*

Table of Contents

1. Product Overview
2. Monorepo Structure
3. Tech Stack
4. Database Schema
5. Authentication System
6. API Routes Reference
7. Voice Call Pipeline Architecture
8. Provider Integrations
9. Tool Execution System
10. Scheduling & Booking System
11. Knowledge Base (RAG) System
12. Campaigns & Batch Calling
13. Caller Memory System
14. Gender / Voice Analysis Service
15. Admin Panel & Billing
16. Deployment Architecture
17. Key Environment Variables
18. Development Workflow
19. Critical Gotchas & Known Issues
20. Current State & Outstanding Work

1. Product Overview

SnapServe (marketed to end-users) / VoiceOrch (internal/platform name) is a SaaS platform for AI voice agent orchestration.

Business Model

- Users bring their own provider API keys (OpenAI, Twilio, ElevenLabs, etc.)
- Platform handles the orchestration layer: routing audio, managing call state, executing tools, logging, billing
- Platform charges an orchestration fee on top; users pay their own provider costs
- Inspired by the open-source Bolna framework

Core Capabilities

Capability	Description
Voice Agents	Create AI agents with configurable ASR + LLM + TTS + Telephony pipeline
Inbound Calls	Receive calls via Twilio/Plivo/VoBiz webhooks
Outbound Calls	Single calls or bulk campaigns with retry logic
Web Calls	Browser-based calls via WebSocket (no telephony required)
Scheduling	AI agent books appointments into Google Calendar live on calls
Knowledge Base	Agent retrieves facts from crawled/uploaded documents
Caller Memory	Agent remembers past interactions with the same phone number
Multilingual	Mid-call language switching with automatic TTS voice swap
Dispositions	Post-call structured classification of call outcome

2. Monorepo Structure

```
voiceorch/
% % %  artifacts/                # Deployable applications
```

```
%    % % %  api-server/            # Core backend (Node.js + Express 5)
%    % % %  dashboard/             # Main SaaS frontend (React + Vite)
%    % % %  admin/                 # Internal admin panel (React + Vite)
%    % % %  family-app/            # Family/Insurance agent portal (web)
%    % % %  family-mobile/         # Family agent mobile app (Expo)
%    % % %  gender-service/        # Python microservice: gender/voice detection
%    % % %  pitch-deck/            # Interactive slide deck (React)
%    % % %  mockup-sandbox/        # UI component preview server (dev tool)
%
% % %  lib/                       # Shared packages (compiled, imported by artifacts)
%    % % %  db/                    # Drizzle ORM schema + migrations + DB client
%    %    % % %  src/schema/       # All table definitions (one file per domain)
%    %    % % %  drizzle/          # Migration SQL files + _journal.json
%    % % %  api-spec/              # OpenAPI 3.0 spec (source of truth for all contracts)
%    %    % % %  openapi.yaml
%    % % %  api-zod/               # Auto-generated Zod schemas (from OpenAPI, do not edit)
%    % % %  api-client-react/      # Auto-generated React Query hooks (from OpenAPI, do not edit)
%
% % %  scripts/                   # One-off utility scripts
% % %  pnpm-workspace.yaml        # Workspace config + catalog pins
% % %  tsconfig.base.json         # Shared TypeScript config
% % %  artifacts/api-server/Dockerfile   # Multi-stage Docker build for Railway
```

Package Names

Directory	Package Name
artifacts/api-server	@workspace/api-server
artifacts/dashboard	@workspace/dashboard
artifacts/admin	@workspace/admin
lib/db	@workspace/db
lib/api-spec	@workspace/api-spec
lib/api-zod	@workspace/api-zod
lib/api-client-react	@workspace/api-client-react

3. Tech Stack

Layer	Technology
Runtime	Node.js 24, TypeScript 5.9
Package manager	pnpm 10 (workspaces)
Backend framework	Express 5
Database	PostgreSQL + Drizzle ORM
Validation	Zod (v4), drizzle-zod
Frontend	React 18 + Vite 7 + Wouter (routing)
UI components	shadcn/ui (Radix UI + Tailwind)
Data fetching	TanStack Query v5
Charts	Recharts
API codegen	Orval (OpenAPI !' React Query hooks + Zod)
Backend build	esbuild (outputs CJS bundle to dist/)
Caching / rate-limit	Upstash Redis (REST API)
Payments	Razorpay
Mobile	Expo (React Native)
Python service	FastAPI + librosa + SpeechBrain

4. Database Schema

> *All tables are in the public schema of PostgreSQL.*
> *Schema files live in lib/db/src/schema/. The DB client is exported from @workspace/db.*

4.1 Core Platform Tables

Table	Purpose
users	Platform users. clerk_id stores Google OAuth sub (column repurposed — not actually Clerk). email, name from Google.
providers	User-supplied API keys (encrypted AES-256-GCM). One row per key per provider type (ASR/LLM/TTS/Telephony).

agents	Agent configurations. Contains all pipeline settings.
calls	Every call record. execution_id links to in-memory session.
phone_numbers	Inbound/outbound phone numbers linked to agents.
webhook_endpoints	User-configured webhooks for call events.
platform_settings	Key-value config (managed keys, rate-limit rows, etc.).
activity_logs	Audit log of user actions.
wallet_transactions	Billing: every debit/credit with reason.
usage_events	Per-call minute-level usage events for billing.

4.2 Agent Extended Tables

Table	Purpose
agent_knowledge_sources	M:N join between agents and knowledge_sources.
agent_dispositions	Per-agent disposition taxonomy (schema for how to classify calls).
agent_variables	Custom prompt variables per agent.
prompt_templates	Reusable prompt templates.

4.3 Call Extended Tables

Table	Purpose
call_events	Granular in-call events (STT results, tool calls, language switches).
call_dispositions	Post-call disposition classification result.
call_transfers	Record of call transfers (cold/warm/agent).
execution_logs	Low-level execution trace per call (component, latency, request/response).
call_bookings	Bookings created via call (links call !’ scheduling booking).
call_callbacks	Scheduled callback requests made during a call.
call_retries	Retry state for campaign calls.

4.4 Campaign Tables

Table	Purpose
campaigns	Outbound calling campaign definition.
campaign_batches	A batch of leads within a campaign.
campaign_leads	Individual leads with contact info, status, attempt count.
incoming_calls	Inbound call routing rules.
scheduled_callbacks	Callbacks scheduled from campaign/call flows.
lead_timeline	Per-lead event timeline.
lead_notes / lead_tasks / lead_tags	Lead CRM data.
lead_sources / lead_transfers	Lead routing metadata.
workspace_dnc	Do-Not-Call list per workspace.
lead_interaction_summaries	AI-generated summaries per lead.
batch_daily_metrics / batch_health_snapshots	Campaign analytics.
batch_import_runs	CSV import job tracking.

4.5 Scheduling Tables

Table	Purpose
scheduling_calendars	One calendar per agent. Links agent !’ Google account !’ timezone.
event_types	Bookable meeting types (name, duration, buffers, capacity).
availability_schedules	Weekly availability windows per calendar.
availability_overrides	Date-specific overrides (block a day, extend hours).
bookings	Confirmed appointments. Stores Google Calendar event ID + Meet link.
google_calendar_connections	OAuth tokens per Google account (encrypted).

4.6 Knowledge Base Tables

Table	Purpose
knowledge_bases	KB container (linked to agent).
knowledge_sources	A source: website URL, uploaded file, or FAQ.
knowledge_entries	Individual Q&A or fact entries.

| kb_documents | Uploaded file metadata. |
| kb_chunks | Document chunks (4000-char splits). |

4.7 Caller Memory Tables

Table	Purpose
caller_profiles	Unified caller profile per phone number.
caller_facts	Atomic facts (name, preferences) with supersession logic.
caller_sentences	Every utterance ever recorded for this caller.
caller_call_archive	Summary of each past call with sentiment.
caller_fact_sentence_links	Which sentences sourced which facts.
caller_episode_fact_links	Which facts are active in which call episodes.

4.8 Other Tables

Table	Purpose
voice_prints	SpeechBrain ECAPA-TDNN speaker embeddings (JSON).
platform_listed_voices	Curated voice library for the UI picker.
ambient_sounds	Background audio tracks agents can play.
user_api_keys	User-generated API keys for programmatic access.
user_google_oauth_config	Per-user Google OAuth app override.
meta_integrations / meta_leads	Meta (Facebook) lead integration.
vobiz_accounts / vobiz_kyc_sessions	VoBiz telephony KYC tracking.
usage_spike_notifications / usage_spike_dismissals	Admin usage alert system.
admin_alerts / alert_notification_log	Internal platform alerts.
simulation_runs / simulation_messages / simulation_test_cases	Agent simulation/testing (local only — not on Railway).
provider_pricing / provider_minute_rates / provider_allocations	Provider cost tracking.
verified_phone_destinations	Verified numbers for trial accounts.

4.9 Migration Strategy

- Migrations live in lib/db/drizzle/ as numbered SQL files
- lib/db/drizzle/meta/_journal.json is the Drizzle migration journal
- migrate(db, { migrationsFolder }) runs at API server startup (auto-applies pending migrations)
- Dev workflow: pnpm --filter @workspace/db run push (drizzle-kit push to dev DB)
- Prod workflow: Commit a new migration SQL file + journal entry !' deploy !' migrations auto-run

> *Warning: Never use drizzle-kit push against production. Add a numbered SQL file instead.*

---

5. Authentication System

End-User Auth (Dashboard)

- Google OAuth (platform owns the Google OAuth app — no Clerk)
- Flow: GET /api/auth/google/connect !' Google consent !' GET /api/auth/google/callback !' issues JWT !' delivers via URL fragment #session=<jwt>
- Session token: HS256 JWT signed with SESSION_SECRET, 30-day TTL
- Payload: { sub: googleSub, email, name }
- Frontend stores in localStorage["voiceorch:session"]
- Every API request sends Authorization: Bearer <jwt> header
- SSE endpoints accept ?token=<jwt> query param
- attachSession middleware parses the JWT and sets req.clerkUserId (= Google sub) + req.sessionPayload
- requireAuth middleware gates protected routes
- getAuth(req) shim in lib/auth-compat.ts keeps ~20 route files backward-compatible

> *Note: The users.clerk_id column stores the Google sub — the column was repurposed and never renamed.*

Admin Auth

- Separate from user auth
- x-admin-token: <ADMIN_TOKEN> header
- requireAdmin middleware gates all /api/admin/* routes
- ADMIN_TOKEN env var (Railway's is different from Replit's)

Provider API Key Encryption

- All provider api_key values are AES-256-GCM encrypted at rest
- Key derived from SESSION_SECRET
- encryptKey(secret, key) / decryptKey(secret, key) in lib/crypto.ts
- Transparent fallback: plaintext keys (pre-encryption era) pass through decryptKey unchanged

6. API Routes Reference

Base path: /api
All routes are in artifacts/api-server/src/routes/

6.1 Agent Routes (`routes/agents.ts`)

Method	Path	Description
GET	/agents	List all agents for user
POST	/agents	Create agent
GET	/agents/:id	Get agent detail
PATCH	/agents/:id	Update agent
DELETE	/agents/:id	Delete agent
PATCH	/agents/:id/toggle	Toggle active/inactive
POST	/agents/:id/call	Initiate outbound call
GET	/agents/:id/versions	List agent versions
POST	/agents/:id/versions	Publish a version snapshot
GET	/agents/:id/versions/:vid	Get a specific version
POST	/agents/:id/versions/:vid/restore	Restore agent to version

6.2 Call Routes (`routes/calls.ts`)

Method	Path	Description
GET	/calls	List calls (filterable)
GET	/calls/:id	Get call detail + transcript
PATCH	/calls/:id/end	End call (auto-computes duration, updates metrics)
GET	/calls/:id/events	Stream call events (SSE)

6.3 Telephony Webhooks

Method	Path	Description
POST	/webhooks/twilio/:agentId	Twilio inbound call webhook
POST	/webhooks/twilio/:agentId/stream	Twilio WebSocket stream handler
POST	/webhooks/plivo/:agentId	Plivo inbound call webhook
POST	/webhooks/vobiz/:agentId	VoBiz inbound call webhook
GET/POST	/webcall/:agentId	Browser WebSocket call

6.4 Provider Routes (`routes/providers.ts`)

Method	Path	Description
GET	/providers	List providers (masked keys)
POST	/providers	Add provider key
PATCH	/providers/:id	Update provider
DELETE	/providers/:id	Delete provider

6.5 Scheduling Routes (`routes/scheduling/`)

Method	Path	Auth	Description
GET/POST	/scheduling/calendars	User	List/create calendars
PATCH/DELETE	/scheduling/calendars/:id	User	Update/delete calendar
GET/POST	/scheduling/event-types	User	List/create event types
PATCH/DELETE	/scheduling/event-types/:id	User	Update/delete event type
GET/POST	/scheduling/availability	User	Manage availability schedules
GET/POST	/scheduling/bookings	User	List/create bookings
PATCH	/scheduling/bookings/:id	User	Update booking (confirm/cancel)
GET	/scheduling/slots	User	Query available slots

GET	/scheduling/google/connect	User	Start Google OAuth for calendar
GET	/scheduling/google/callback	None	OAuth callback (uses state)
GET/DELETE	/scheduling/google/connections	User	Manage Google connections
POST	/scheduling/public/bookings	None	Public booking creation
GET	/scheduling/public/event-types/:slug	None	Public event type info
GET	/scheduling/public/slots	None	Public slot query

6.6 Campaign Routes (`routes/campaigns/`)

Method	Path	Description
GET/POST	/campaigns	List/create campaigns
PATCH/DELETE	/campaigns/:id	Update/delete campaign
POST	/campaigns/:id/launch	Launch campaign
POST	/campaigns/:id/pause	Pause campaign
GET/POST	/campaign-batches	List/create batches
POST	/campaign-batches/:id/import	Import CSV leads
GET	/campaign-leads	List leads (filterable)

6.7 Analytics & Platform

Method	Path	Description
GET	/analytics/overview	Dashboard stats (calls today, success rate, avg duration)
GET	/analytics/call-volume	Time-series call volume
GET	/analytics/status-breakdown	Pie chart data
GET	/voices	List available TTS voices per provider
GET	/ambient-sounds	List ambient sound tracks
GET	/knowledge-bases	List KB entries
GET	/health	Health check (returns 200)

6.8 Admin Routes (`routes/admin/`, requires `x-admin-token`)

Path	Description
/admin/users	User list + wallet adjustments
/admin/usage	Platform-wide usage metrics
/admin/platform-settings	Manage managed API keys (e.g., Groq key)
/admin/voices	Manage platform-listed voices
/admin/alerts	Usage spike alerts
/admin/transactions	All wallet transactions

7. Voice Call Pipeline Architecture

7.1 Overview

Two separate but parallel pipeline implementations:

File	Handles
src/orchestration/pipeline.ts	Telephony calls (Twilio, Plivo, VoBiz)
src/orchestration/telephony/webcall.ts	Browser-based calls via WebSocket

> *Critical: Any feature added to pipeline.ts must be manually replicated in webcall.ts. There is no shared base class.*

7.2 Audio Processing Loop

```
Caller Audio (mu-law 8kHz or PCM 16kHz)
    !“
Noise Cancellation (optional, RNNoise)
    !“
VAD (Voice Activity Detection) — detects speech onset/silence
    !“
STT Adapter (streams audio !’ text)
    !“
processTurn()
    % % %  KB context retrieval (if agent has KB)
    % % %  Caller Memory injection (prior facts + episode summary)
    % % %  Language detection & system prompt language injection
```

```
% % %  LLM Adapter (text !’ response or tool call)
        !“
    TTS Adapter (text !’ audio chunks, streamed back)
        !“
    Caller Speaker (audio back to caller)
```

7.3 Turn Lifecycle

1. onAudioIn() — receives raw audio frame from telephony provider
2. Passed to audioPool worker for noise cancellation + VAD analysis
3. If VAD detects speech: buffer frames and send to STT
4. On STT result: check barge-in threshold if agent is currently speaking
5. If barge-in triggered: stop TTS, drain pending audio queue
6. processTurn(transcript):
   - %æ Build message history (respecting contextWindowTurns limit)
   - %æ If tokenSummarizationEnabled: compress old turns via LLM before sending
   - %æ If stagedPromptEnabled: switch system prompt based on conversation stage
   - %æ Call LLM with tools schema
7. LLM response handling:
   - %æ Text: stream to TTS !’ stream audio back to caller
   - %æ Tool call: execute tool, inject result, call LLM again
8. Turn complete: save transcript + events to DB

7.4 Key Pipeline Settings (per Agent)

Setting	Effect
firstSpeaker	assistant (agent speaks first) or user (wait for caller)
silenceTimeoutSeconds	Max silence before ending call
backchannelingEnabled	Agent makes "mm-hmm" sounds while listening
contextWindowTurns	Max N turns kept in LLM history
tokenSummarizationEnabled	Compress history when it exceeds threshold
stagedPromptEnabled	Multi-stage system prompt (e.g., intro !’ discovery !’ close)
language / asr_language	Primary language
asrSmartEndpointing	How aggressively to detect end-of-turn
noiseCanecllationEnabled	Run RNNoise on caller audio

7.5 Multilingual Support

- azure.ts STT adapter uses continuous Language Identification (LID) mode when candidateLocales.length > 1
- When LID detects a language switch mid-call, pipeline.ts and webcall.ts inject [ACTIVE LANGUAGE] Respond ONLY in {langName} as a system prompt suffix
- TTS voice swaps to the language-appropriate voice from the agent's multilingualLanguages config

8. Provider Integrations

8.1 STT (ASR) Providers

Provider	File	Notes
Azure	adapters/stt/azure.ts	Continuous LID for multilingual; every frame must include X-RequestId header
Deepgram	adapters/stt/deepgram.ts	Streaming
AssemblyAI	adapters/stt/assembly.ts	Streaming
Sarvam	adapters/stt/sarvam.ts	Good for Indic languages
Cartesia	adapters/stt/cartesia.ts	
Whisper	adapters/stt/whisper.ts	Batch (not streaming)

8.2 LLM Providers

Provider	Notes
OpenAI	Default; GPT-4o, GPT-4o-mini, etc.
Anthropic	Claude 3.x series
Google	Gemini series
Groq	Ultra-low latency; used for managed KB queries (Qwen-32B)
DeepSeek	DeepSeek-V3, R1
Mistral	

Cohere	
Qwen	Via Groq API or direct
Sarvam	

Managed Key: Platform provides a managed_qwen_key (Groq API key) seeded from GROQ_API_KEY env var into platform_settings at startup. Admin panel can override it.

8.3 TTS Providers

Provider	Notes
ElevenLabs	High-quality; many voices
Cartesia	Low latency
Sarvam	Indic language voices
OpenAI	TTS-1, TTS-1-HD
Azure	Neural voices
Deepgram	Aura voices
AWS Polly	

8.4 Telephony Providers

Provider	Protocol	Notes
Twilio	<Connect><Stream> WebSocket	Base64 mu-law 8kHz frames
Plivo	AudioSocket WebSocket	Binary mu-law 8kHz
VoBiz	Raw WebSocket	PCM 8kHz; has KYC/account management

9. Tool Execution System

File: artifacts/api-server/src/services/tool-executor.ts

9.1 Available Tools

Tool Name	Function	Description
book_appointment	native_booking	Books a slot in the agent's calendar
check_availability	native_check_availability	Lists open slots for a date range
list_bookings	native_list_bookings	Returns caller's existing bookings
reschedule_booking	native_reschedule	Moves an existing booking to a new time
cancel_booking	native_cancel	Cancels a booking by ID
transfer_cold	transfer_cold	Blind call transfer to a number
transfer_warm	transfer_warm	Warm handoff with hold message
transfer_agent	transfer_agent	Hand off to another AI agent
webhook	webhook	POST to user-configured URL
send_sms	send_sms	Send SMS via telephony provider
end_call	end_call	Hang up the call

9.2 Tool Resolution

matchToolByName(toolName, tools) — exported from tool-executor.ts. Matches the LLM's function call name to the agent's configured tool definition.

9.3 Booking Tool Architecture

All booking tools share the same resolution chain:

```
Agent !’ Assigned Calendar (scheduling_calendars.agent_id)
    !’ Event Types (event_types.calendar_id)
    !’ Google Account (google_calendar_connections.id)
    !’ Slot Engine !’ Advisory Lock !’ Create Booking !’ GCal Event + Email + Webhook
```

Important: The booking execution path is triplicated in the codebase:

1. tool-executor.ts — for in-call tool execution
2. routes/scheduling/public.ts — for unauthenticated public booking page
3. routes/scheduling/bookings.ts — for authenticated API calls

Any change to booking logic (notifications, webhooks, GCal sync) must be applied in all three places.

10. Scheduling & Booking System

10.1 How It Fits Together

```
scheduling_calendars (one per agent)
    % % %  event_types[] (e.g., "Discovery Call", "Demo")
    %        % % %  availability_schedules[] (Mon 9–5, Wed 10–4, ...)
    %        % % %  availability_overrides[] (Dec 25: blocked)
    %        % % %  bookings[] (confirmed appointments)
    % % %  google_calendar_connections (OAuth tokens, encrypted)
```

10.2 Slot Engine

File: services/scheduling/slot-engine.ts

1. Fetches recurring availability_schedules for the event type's calendar
2. Applies availability_overrides (additions/subtractions for specific dates)
3. Uses date-fns-tz for all timezone conversions
4. Subtracts existing bookings plus their buffer periods
5. Returns list of available ISO timestamp slots

10.3 Double-Booking Prevention

Uses PostgreSQL advisory locks:

```
SELECT pg_try_advisory_xact_lock(hashtext('calendar:' || calendarId || ':' || slotStart))
```

Runs inside a transaction. If lock fails !' slot taken !' return 409.

10.4 Google Calendar Integration

- Platform owns ONE Google OAuth app (GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET)
- Users click "Connect with Google" — no dev setup required
- Multiple Google accounts per user supported
- Each calendar can be linked to a specific Google account via google_calendar_connection_id
- Tokens encrypted at rest; auto-refreshed on expiry
- Creates Google Meet links automatically (enableGoogleMeet: true)

10.5 Public Booking Page

Route: /book/:slug (frontend)
API: GET /api/scheduling/public/event-types/:slug, GET /api/scheduling/public/slots, POST /api/scheduling/public/bookings
Completely unauthenticated — no login required for attendees.

10.6 Notifications

- Email: Nodemailer via Gmail (GMAIL_USER + GMAIL_APP_PASSWORD env vars, optional — silently skipped if absent)
- Webhooks: dispatchSchedulingWebhooks() sends scheduling.booking.created/confirmed/cancelled/rescheduled events with HMAC signature (X-VoiceOrch-Signature)
- Self-service tokens: Each booking email contains signed cancel/reschedule links (no login required)

11. Knowledge Base (RAG) System

11.1 Architecture

Unlike traditional vector-DB RAG, this system uses PostgreSQL Full-Text Search + LLM-as-retriever:

1. Indexing: Documents are chunked (4000 chars) and stored in kb_chunks. A GIN index on knowledge_entries enables FTS.
2. Retrieval: At call time, the agent's question triggers a FTS query against knowledge_entries returning up to 200 matching entries.
3. Synthesis: The 200 entries are passed directly into the LLM context (Qwen-32B via Groq managed key). No embeddings, no pgvector.

11.2 Source Types

Type	How it works
Website	URL crawled at indexing time, content extracted
File Upload	PDF, DOCX, TXT, MD uploaded and chunked

| FAQ | Manual Q&A pairs entered in the UI |

11.3 Agent Linking

agent_knowledge_sources (M:N join) links an agent to multiple knowledge sources. An agent can draw from multiple KBs simultaneously.

12. Campaigns & Batch Calling

12.1 Campaign Structure

```
Campaign
    % % %  CampaignBatch (a group of leads, one at a time)
            % % %  CampaignLead[] (individual contacts)
```

12.2 Lead Import

- CSV upload to POST /campaign-batches/:id/import
- User maps CSV columns to standard fields (name, phone, custom fields)
- Custom fields stored as JSON in lead_fields

12.3 Execution

- campaignCaller.ts — orchestrates concurrent outbound calls
- Respects max_concurrent_calls per agent
- Retry logic: configurable attempts + delay; stores retry state in call_retries
- Disposition-aware: uses call outcome to determine whether to retry

12.4 DNC (Do-Not-Call)

workspace_dnc table stores blocked numbers per workspace. Checked before each campaign call.

13. Caller Memory System

File: services/callerMemory.ts

Three-layer Graphiti-style memory per phone number:

Layer	Table	Description
L0 Facts	caller_facts	Atomic facts: name, preferences, objections. Conflict resolution via supersession (new fact overwrites old).
L1 Episodes	caller_call_archive	Summary of each past call with sentiment score and key outcomes.
L2 Sentences	caller_sentences	Every utterance ever recorded (used for fact sourcing).

Resume Logic

If a call drops mid-conversation, the system loads the "tail" of the last session's sentences and injects them as context so the agent can resume naturally.

Injection Format

At each call start, the memory system injects a formatted context block into the system prompt:

```
[CALLER MEMORY]
Name: Rajesh Kumar
Last called: 2026-05-14 (interested in family floater plan)
Known preferences: prefers Hindi, budget  ¹15,000/year
Active facts: 3
```

14. Gender / Voice Analysis Service

14.1 Service

- Technology: Python, FastAPI, librosa, SpeechBrain ECAPA-TDNN
- Runs on: Port 5000 (separate process, artifacts/gender-service)
- Endpoint: POST /detect-gender — accepts WAV audio, returns { gender: "male"|"female", confidence: float }

14.2 How It's Used in Calls

1. First ~2 seconds of caller audio is buffered

2. Converted to 16kHz WAV and sent to gender service
3. Result stored in call session memory
4. Agent uses it to select appropriate salutation (Mr./Ms.)

14.3 Speaker Recognition (ECAPA-TDNN)

- Speaker embeddings stored in voice_prints.embedding (JSON text)
- Used to identify returning callers by voice even if they call from a different number
- Replaces the previously-used Azure Speaker Recognition API

14.4 Import Quirk

librosa must be lazy-imported inside the request handler, NOT at the top level. Top-level import blocks uvicorn from binding its port in production.

15. Admin Panel & Billing

15.1 Admin Panel Features

- Overview: Total revenue, active users, calls today, platform success rate
- Users: List all users, adjust wallet balance, override concurrent call limits
- Transactions: Full wallet transaction history across all users
- Platform Settings: Set managed API keys (Groq/Qwen), toggle features
- Voices: Manage the platform-listed voice library
- Usage Alerts: Review and dismiss usage spike notifications
- Health: Failed call reason aggregation, provider error rates

15.2 Wallet System

- Each user has a wallet in USD cents
- Debited per call based on minute-rate × duration
- provider_pricing / provider_minute_rates tables store rates per provider combination
- Top-up via Razorpay (RAZORPAY_KEY_ID + RAZORPAY_KEY_SECRET)
- Autopay: wallet auto-refills when balance drops below threshold
- Welcome credit: new users get $5 on first login (userProvisioning.ts)

15.3 Rate Limiting

- Per-user rate limits enforced via Upstash Redis
- Short-lived rate-limit rows in platform_settings auto-cleaned by services/rate-limit-cleanup.ts
- Any code writing transient keys to platform_settings must call registerTransientKey({ prefix, windowMs }) at startup — do NOT add a separate cleanup loop

16. Deployment Architecture

16.1 Overview

```
GitHub (main branch)
    !“ push
Railway (auto-deploy)
    !“ Docker build (Dockerfile in artifacts/api-server/)
    !“ pnpm install
    !“ Build dashboard (Vite !’ dist/public/)
    !“ Build admin (Vite !’ dist/public/)
    !“ Build api-server (esbuild !’ dist/index.mjs)
    !“ Deploy runtime container
    !“ Server starts !’ migrate() runs !’ app live
```

16.2 Dockerfile (Multi-stage)

```
# Stage 1: Builder (Node 22 slim)
# - Installs all pnpm dependencies
# - Builds dashboard, admin (Vite)
# - Builds api-server (esbuild)

# Stage 2: Runtime (Node 22 slim)
# - Production dependencies only
# - Copies built artifacts from builder
# - Copies drizzle migrations
# - Exposes port 8080
# - CMD: node --enable-source-maps artifacts/api-server/dist/index.mjs
```

16.3 Service Routing

The API server serves everything on port 8080:

Path	Served By
/api/*	Express route handlers
/admin-ui/*	Static files from artifacts/admin/dist/public
/*	Static files from artifacts/dashboard/dist/public
* (SPA fallback)	index.html for client-side routing

16.4 Replit (Development)

Each artifact has its own workflow:

- artifacts/api-server !’ port 8080
- artifacts/dashboard !’ port 23183
- artifacts/admin !’ port varies
- artifacts/gender-service !’ port 5000

Replit's shared proxy routes traffic: /api !’ 8080, / !’ 23183, etc.

Important: Replit and Railway are completely separate environments. fetch_deployment_logs in Replit only shows Replit's health-check pings — not Railway production logs. Check Railway console directly for production errors.

17. Key Environment Variables

Required (Production)

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
SESSION_SECRET	Signs JWTs + derives AES key for provider key encryption
GOOGLE_CLIENT_ID	Google OAuth app client ID (shared: auth + calendar)
GOOGLE_CLIENT_SECRET	Google OAuth app client secret
ADMIN_TOKEN	Bearer token for admin API routes
GROQ_API_KEY	Seeds the managed Qwen/Groq key for KB queries

Payments

Variable	Purpose
RAZORPAY_KEY_ID	Razorpay payment gateway
RAZORPAY_KEY_SECRET	Razorpay secret

Caching / Rate Limiting

Variable	Purpose
UPSTASH_REDIS_REST_URL	Upstash Redis REST endpoint
UPSTASH_REDIS_REST_TOKEN	Upstash auth token

Telephony

Variable	Purpose
VOBIZ_AUTH_ID	VoBiz account ID
VOBIZ_AUTH_TOKEN	VoBiz auth token

AI Services

Variable	Purpose
SARVAM_API_KEY	Sarvam ASR/TTS (Indic languages)
CARTESIA_API_KEY	Cartesia TTS

Email (Optional)

Variable	Purpose

| GMAIL_USER | Gmail sender address for booking notifications |
| GMAIL_APP_PASSWORD | Gmail app password |

Object Storage

Variable	Purpose
DEFAULT_OBJECT_STORAGE_BUCKET_ID	Replit object storage bucket
PRIVATE_OBJECT_DIR	Private storage path prefix
PUBLIC_OBJECT_SEARCH_PATHS	Public storage search paths

GitHub

Variable	Purpose
GITHUB_PAT	GitHub Personal Access Token (for pushing code from Replit)

18. Development Workflow

Initial Setup

```
pnpm install
# Set DATABASE_URL in .env
pnpm --filter @workspace/db run push    # Apply schema to dev DB
```

Running Locally

Use Replit workflows (not pnpm dev at root — that script doesn't exist):

- Start artifacts/api-server: API Server workflow
- Start artifacts/dashboard: web workflow

After Changing the OpenAPI Spec

```
# Always run this first after editing lib/api-spec/openapi.yaml:
pnpm --filter @workspace/api-spec run codegen

# Then typecheck:
pnpm run typecheck:libs
```

After Changing DB Schema

```
# Dev (direct push to dev DB):
pnpm --filter @workspace/db run push

# Production (create migration file):
# 1. Add numbered SQL file to lib/db/drizzle/
# 2. Add entry to lib/db/drizzle/meta/_journal.json
# 3. Push to GitHub !’ Railway auto-applies on next deploy
```

Typechecking

```
pnpm run typecheck:libs    # Compile lib packages
pnpm run typecheck         # Full check: libs + all artifacts
```

Pushing to Production

```
# All code changes must go to GitHub main branch
# Railway auto-deploys on push

# For large files, use the Python urllib push script pattern:
python3 - <<'EOF'
import json, base64, urllib.request, os
pat = os.environ["GITHUB_PERSONAL_ACCESS_TOKEN"]
# ... (see any prior push script in session history)
EOF
```

After Editing Backend Routes

The API server has no hot-reload — it runs a prebuilt esbuild bundle. After route/middleware edits, restart the API Server workflow. New endpoints will 404 until restarted.

19. Critical Gotchas & Known Issues

Code Architecture

1. Two pipeline implementations — pipeline.ts (telephony) and webcall.ts (browser calls) are entirely separate. Every feature must be added to both.

2. Booking path is triplicated — GCal sync, email notifications, and webhook dispatch are inlined in 3 separate places: tool-executor.ts, routes/scheduling/public.ts, routes/scheduling/bookings.ts. Change all three.

3. drizzle-kit push loops forever on FK names > 63 chars (PostgreSQL truncates them). Also drops raw-SQL-only columns silently. Use migrations for production, not push.

4. Radix Select value="" crashes. Use __none__ as sentinel and convert on onChange/value.

5. Rate-limit cleanup: Any code writing transient rows to platform_settings must call registerTransientKey({ prefix, windowMs }). Do not add separate cleanup loops.

Infrastructure

6. GitHub push required for EVERY production change. Railway deploys from GitHub, not Replit. The local Replit environment and Railway production are completely separate.

7. Generated files must be committed. lib/api-client-react/src/generated/api.schemas.ts and lib/api-zod/src/generated/ are not regenerated during the Railway Docker build. They must be committed to GitHub.

8. ADMIN_TOKEN differs between Replit and Railway. Don't test admin endpoints with Replit's token against the Railway API.

9. Azure STT: Every WebSocket frame (text + binary) must include X-RequestId header. Omitting it causes 1002 close. Batch confidence filter is too aggressive for Indic languages — use RecognitionStatus=Success check instead.

10. librosa in gender service must be lazy-imported (inside the handler function). Top-level import blocks uvicorn from binding its port.

Database

11. users.clerk_id stores Google sub, not a Clerk ID. The column was repurposed and never renamed. All queries using clerkUserId are actually using the Google subject.

12. Migration 0001 (0001_production_schema_sync.sql) adds all columns and tables added post-initial deployment. This was applied to Railway in June 2026.

20. Current State & Outstanding Work

Recently Fixed (June 2026)

- ' Railway build failures resolved (missing matchToolByName export, edit.schema.ts, generated api.schemas.ts)
- ' Multilingual STT: Azure LID continuous mode for multi-language agents
- ' Multilingual LLM drift: [ACTIVE LANGUAGE] system prompt injection in both pipeline paths
- ' Production DB schema sync: Migration 0001 adds 11 missing columns + 5 missing tables to Railway
- ' Booking tools: Event type dropdown now loads correctly (was using et.title instead of et.name)
- ' Booking tools: Reschedule + Cancel now show the event type selector

Known Areas for Improvement

- Simulation tabs — local-only feature (not on GitHub/Railway yet)
- Agent versions — versioning UI exists locally; backend routes exist locally but not deployed
- VoBiz KYC — no GET /accounts endpoint; recovery via scanning KYC sessions
- Speaker recognition — embeddings stored as JSON text (not optimized for similarity search)
- KB search — no vector embeddings; LLM-as-retriever approach may hit context limits at scale
- Booking path triplication — should be refactored into a shared booking service

This document was auto-generated from codebase analysis. For the most current state, always cross-reference with replit.md and the GitHub commit history.